Module 3.2

Google AppEngine

Google App Engine (GAE)

- Google App Engine (often referred to as GAE or simply App Engine) is a cloud computing **platform as a service for developing and hosting web applications in Google-managed data centers.**
- PAAS

-Definition

Google App Engine (GAE)

Key Features-

- Popular languages
- Fully managed
- Powerful application diagnostics
- Application versioning
- Application security
- Services ecosystem

Google App Engine (GAE)

Popular languages

- Supported programming languages-
- Build your application in **Node.js, Java, Ruby, C#, Go, Python, or PHP.**

Fully managed

- A fully managed environment lets you focus on code while App Engine manages infrastructure concerns.

Google App Engine (GAE)

Powerful application diagnostics

- Use Cloud Monitoring and Cloud Logging to monitor the health and performance of your app and Error Reporting to diagnose and fix bugs quickly.

Application versioning

- Easily host different versions of your app, and easily create development, test, staging, and production environments.

Google App Engine (GAE)

Application security

- Help safeguard your application by defining access rules with App Engine firewall and leverage managed SSL(Secure Socket Layer)/TLS (Transport Layer Security) certificates by default on your custom domain at no additional cost.

Services ecosystem

- Tap a growing ecosystem of Google Cloud services from your app including an excellent suite of cloud developer tools.

Google App Engine (GAE)

- Google has hundreds of data centers and has installed more than 460,000 servers worldwide.
- For example, 200 Google data centers are used at one time for a number of cloud applications.
- Data items are stored in text, images, and video and are replicated to tolerate faults or failures.
- Here we discuss Google's App Engine (GAE) which offers a PaaS platform supporting various cloud and web applications.

Google App Engine (GAE)

- Applications are **sandboxed and run across multiple servers**.
- App Engine offers **automatic scaling for web applications**—as the number of requests increases for an application,
- App Engine **automatically allocates more resources** for the web application to handle the additional demand.

-Definition

Architecture

- AppEngine is a platform for developing scalable applications accessible through the Web.
- The platform is logically divided into four major components:

1) Infrastructure,
2) The runtime environment,
3) The underlying storage, and
4) The set of scalable services that can be used to develop applications

Architecture

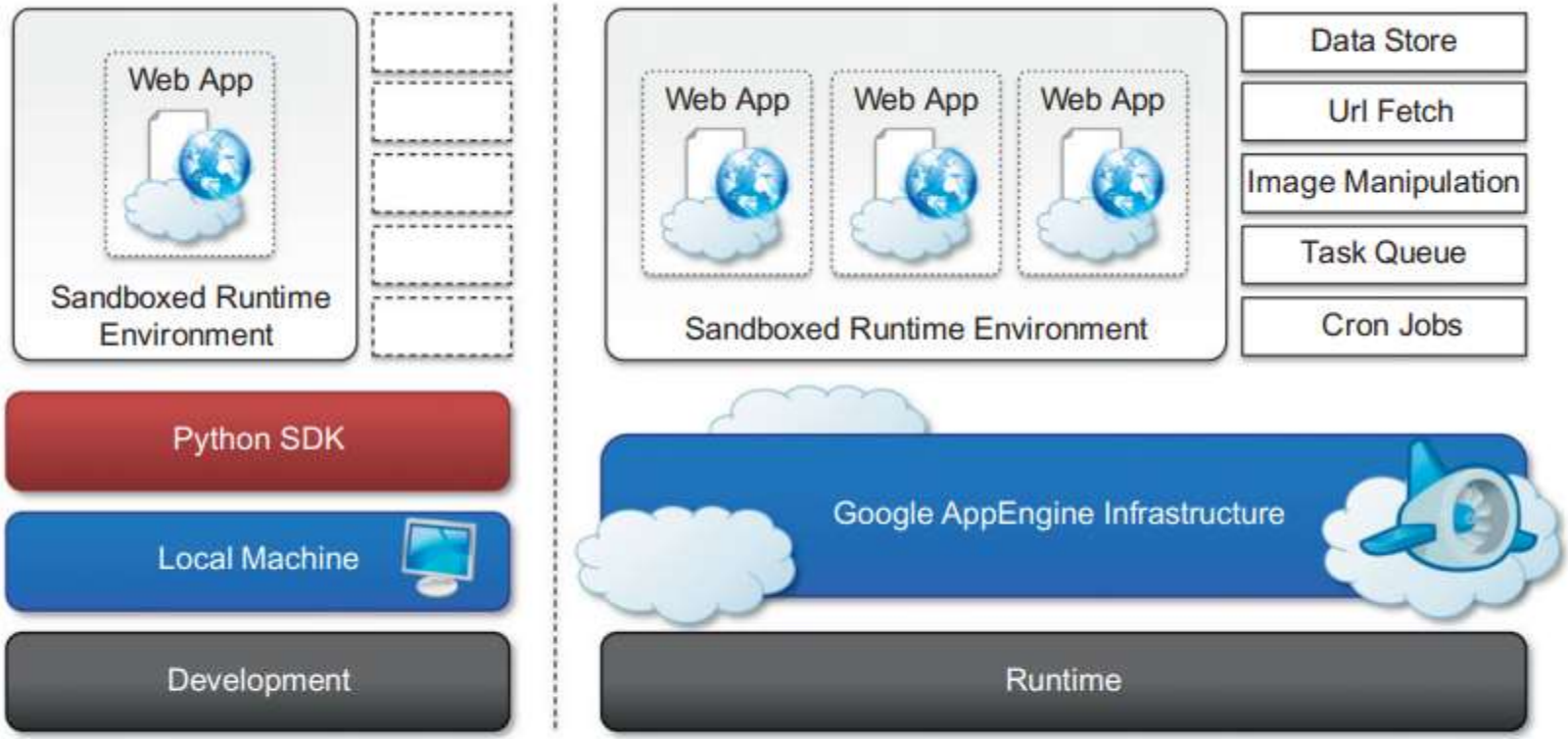


FIGURE 9.2

Google AppEngine platform architecture.

Architecture

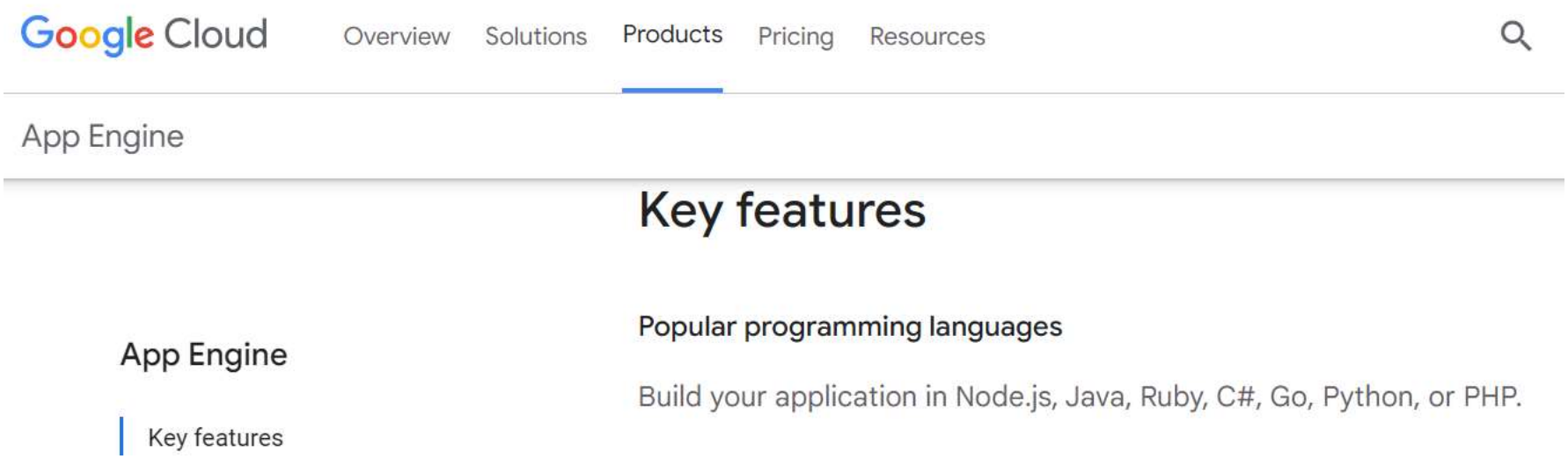
- AppEngine **hosts Web applications, and its primary function is to serve users requests efficiently.**
- To do so, AppEngine's infrastructure takes advantage of **many servers available within Google datacenters.**
- For each HTTP request, **AppEngine locates the servers hosting the application that processes the request**, **evaluates their load, and, if necessary, allocates additional resources** (i.e., servers) or redirects the request to an existing server

Run time Environment

- The runtime environment represents **the execution context of applications hosted on AppEngine.**
- With reference to the **AppEngine infrastructure code, which is always active and running**, **the runtime comes into existence when the request handler starts executing and terminates** once the handler has completed.

Supported runtimes

- Currently, it is possible to develop AppEngine applications using three different languages and related technologies: Java, Python, and Go



Supported runtimes

- Support for Python is provided by **an optimized Python interpreter.**
- The environment supports the Python standard library, but **some of the modules that implement potentially harmful operations have been removed**, and **attempts to import such modules or to call specific methods generate exceptions.**

Supported runtimes

- Developers can use the common tools for Web application development in Java, **such as the Java Server Pages (JSP), and the applications interact with the environment by using the Java Servlet standard.**

Supported runtimes

- **The Go runtime environment allows applications developed with the Go programming language** to be hosted and executed in AppEngine.

Supported runtimes

App Engine

Key features

Documentation

All features

Pricing

Popular languages	Build your application in Node.js, Java, Ruby, C#, Go, Python, or PHP.
Fully managed	A fully managed environment lets you focus on code while App Engine manages infrastructure concerns.

Runtimes

- The App Engine standard environment is based on **container instances running on Google's infrastructure.**
- **Containers are preconfigured with one of several available runtimes.**

Runtimes

- The standard environment makes it easy to build and deploy an application that runs reliably even under heavy load and with large amounts of data.
- Applications run in a secure, sandboxed environment. Your application runs within its own secure, reliable environment that is independent of the hardware, operating system, or physical location of the server.

Instance classes

- **The *instance class* determines the amount of memory and CPU available to each instance, the amount of free quota, and the cost per hour after your app exceeds the free quota.**

- **For all runtime generations, the memory limit includes the memory your app uses along with the memory that the runtime itself needs to run your app.**

- The Java runtimes use more memory to run your app than other runtimes.

Instance classes

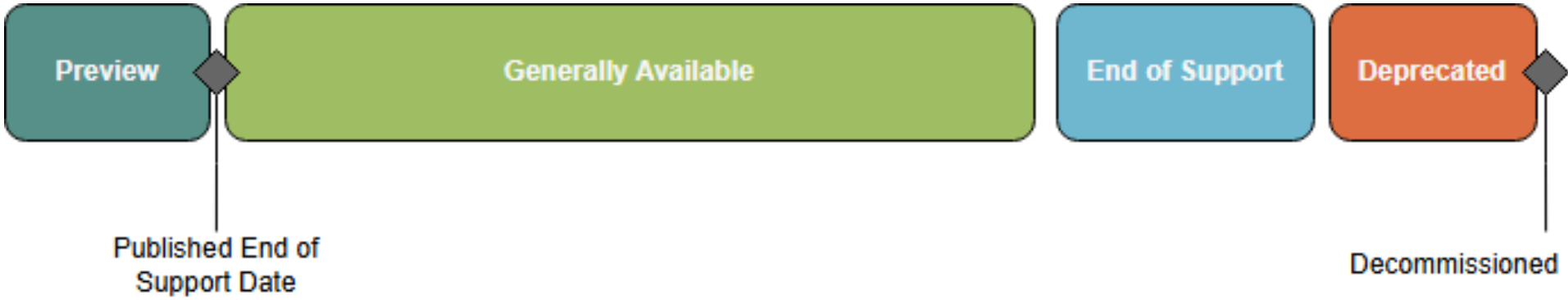
- **The *instance classes*-**

Instance Class	Memory Limit	CPU Limit	Supported Scaling Types
F1 (default)	384 MB	600 MHz	automatic
F2	768 MB	1.2 GHz	automatic
F4	1536 MB	2.4 GHz	automatic
F4_1G	3072 MB	2.4 GHz	automatic
B1	384 MB	600 MHz	manual, basic
B2 (default)	768 MB	1.2 GHz	manual, basic
B4	1536 MB	2.4 GHz	manual, basic
B4_1G	3072 MB	2.4 GHz	manual, basic
B8	3072 MB	4.8 GHz	manual, basic

Runtime Lifecycle

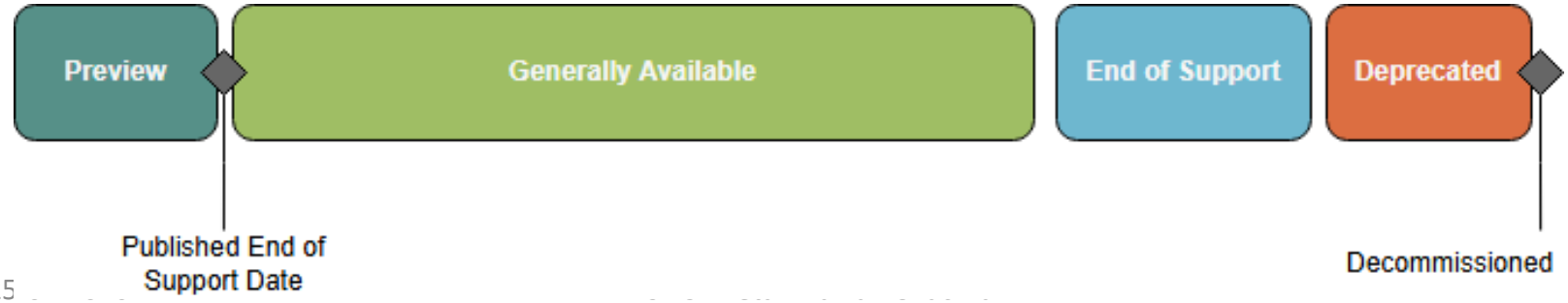
- The App Engine standard environment runtimes use open source components that are maintained by their respective communities.

- The runtimes are identified by their language version, for example, Java 17, Python 3.10, and so forth.



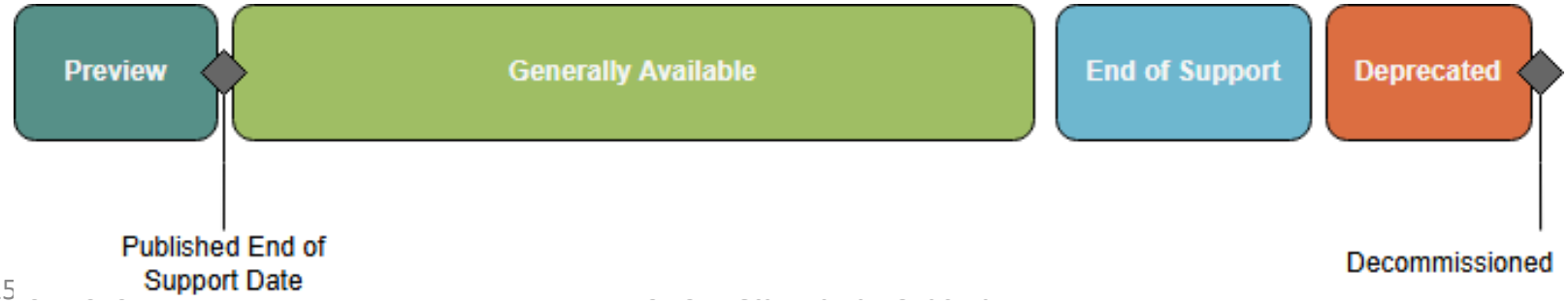
Runtime Lifecycle

- Google provides support for a runtime during General availability (GA).
- During this support window:
  - Runtime components are regularly updated with security and bug fixes.
  - To maintain stability, App Engine avoids implementing changes into the runtime. Breaking changes will be announced in advance on the runtime-specific release notes.



Runtime Lifecycle

- When a language version is no longer actively maintained by the respective community, App Engine will also stop providing maintenance and support for that language runtime.



Runtime Lifecycle

- Google may make changes to any runtime's support schedule or lifecycle in accordance with the terms of your agreement for the use of Google Cloud platform services.

Runtime lifecycle

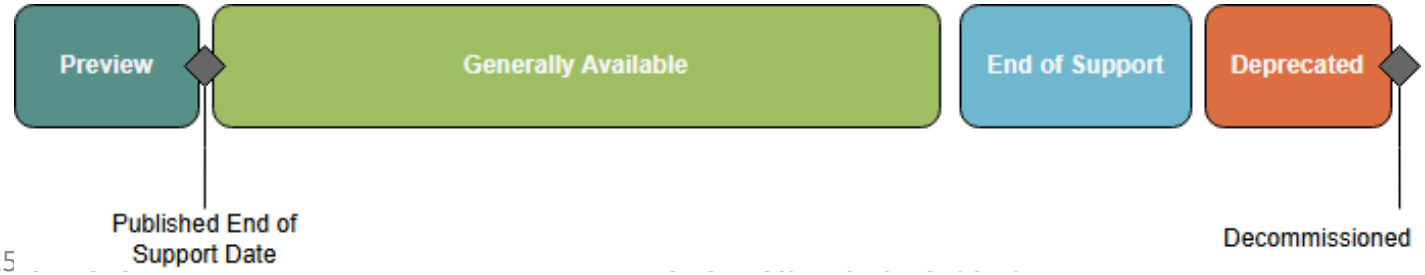
	GA-level support	End of Support	Deprecated	Decommissioned
Creation & redeployment	Yes	No[1]	No	No
Project Configuration Updates	Yes	Yes	No	No
Running existing workloads	Yes	Yes	Yes	No
UI & CLI Warnings	Yes	Yes	No	No
Language patches	Automatic	No automatic updates	No automatic updates	No automatic updates
Patching APIs & SDKs	Automatic	No automatic updates	No automatic updates	No automatic updates
Patching OS	Automatic	No automatic updates	No automatic updates	No automatic updates
Customer Support	GA-level support	No runtime support	No runtime support	No runtime support

Runtime Lifecycle

- Before a runtime reaches the end of support phase as described in the runtimes support schedule, Google will provide a notification to customers.

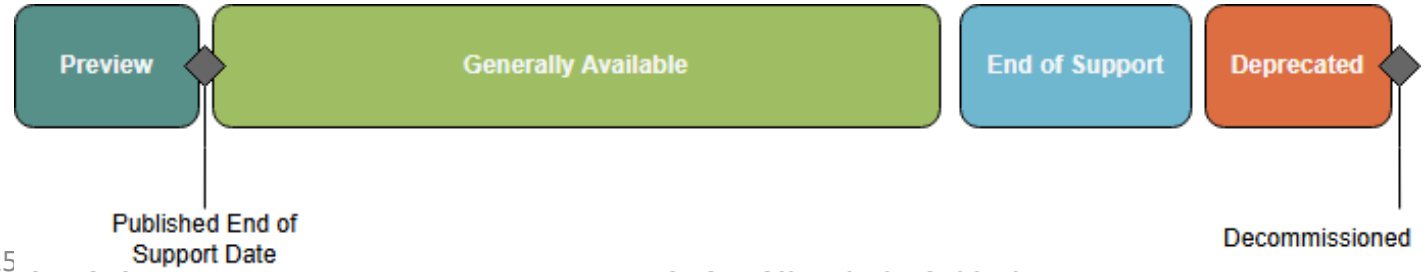
Notification period

- App Engine will begin issuing in-app notifications 90 days before the application reaches end of support.
- Upon notification, you should prepare to upgrade your application to a newer runtime that is supported in the standard environment.



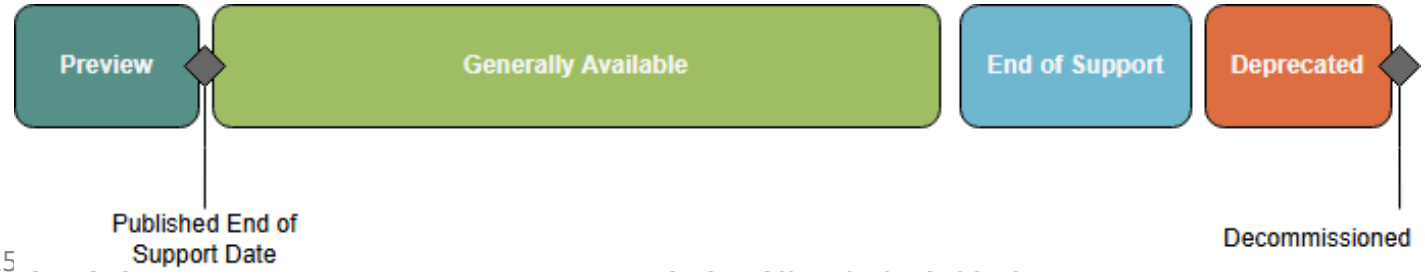
End Of Support

- When runtime components reach the end of support date:
  - Google will no longer apply security updates or patches to components of the runtime environment.
  - Your application will continue to run and receive traffic.
  - You will no longer be able to create and/or update the application on the unsupported runtime.
  - Issues arising from the use of an unsupported runtime will not be eligible for technical support



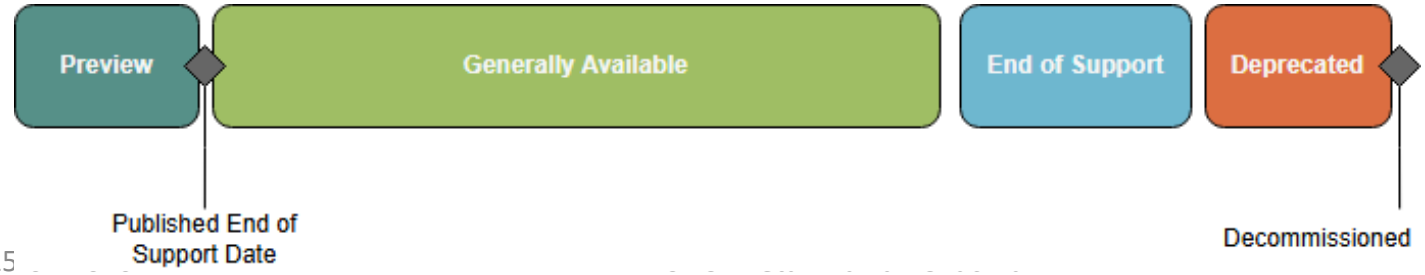
End Of Support

- In certain cases, Google may permit your Organization to re-enable deployments in unsupported runtimes, subject to service specific terms and conditions.
- Deployments for legacy runtimes can be re-enabled using an organization policy.



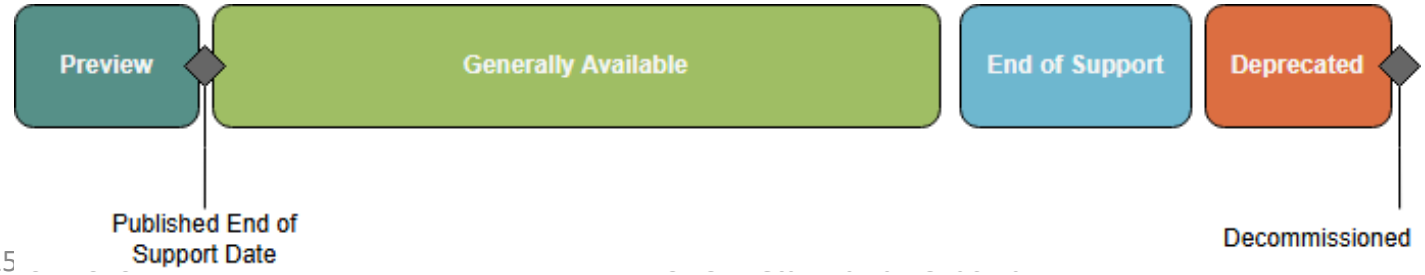
Deprecated

- If Google allowed your Organization to re-enable deployments in an unsupported runtime, Google will remove that ability once the runtime is deprecated.
- Where practicable, Google makes reasonable efforts to notify you in advance of the deprecation by in-app notifications or other means.



Decommissioned

- Applications that continue to use a decommissioned runtime may be disabled without further notice.



Supported runtimes

App Engine

Key features

Documentation

All features

Pricing

Popular languages	Build your application in Node.js, Java, Ruby, C#, Go, Python, or PHP.
Fully managed	A fully managed environment lets you focus on code while App Engine manages infrastructure concerns.

Sandboxing

- **To provide the application environment with an isolated and protected context in which it can execute**
- **without causing a threat to the server and**
- **without being influenced by other applications.**

- **GAE provides applications with a sandbox.**

Sandboxing

- The isolation metaphor is taken from the idea of children who do not play well together, so each is given their own sandbox to play in alone.

Sandboxing

- Currently, AppEngine **supports applications that are developed only with interpreted or managed languages, which by design require a runtime** for translating their code into executable instructions.
- Therefore, **sandboxing is achieved by means of modified runtimes for applications that disable some of the common features normally available with their default implementations**

Sandboxing

- If an application tries to perform any operation that is considered potentially harmful, **an exception is thrown and the execution is interrupted.**

Sandboxing

- Some of the operations that are not allowed in the sandbox include
- **writing to the server's file system**
- **accessing computer through network besides using Mail,UrlFetch, and XMPP(**Extensible Messaging and Presence Protocol)
- **executing code outside the scope of a request**
- **a queued task**
- **processing a request for more than 30 seconds.**

Storage

- AppEngine provides **various types of storage, which operate differently depending on the volatility of the data.**
- There are three different levels of storage: in memory-cache, storage for semistructured data, and long-term storage for static data.

Static file servers

- Web applications are composed of dynamic and static data.
- Dynamic data are a result of the logic of the application and the interaction with the user.

Static file servers

- Static data often are mostly constituted of the **components that define the graphical layout of the application (CSS files, plain HTML files, JavaScript files, images, icons, and sound files) or data files.**
- These files **can be hosted on static file servers, since they are not frequently modified.**

Google Cloud Platform Services

Datastore

- Datastore is a highly scalable NoSQL database for your web and mobile applications.
- **Datastore automatically handles sharding and replication,** providing you with a highly available and durable database that scales automatically to handle your applications' load.
- Datastore provides a **myriad of capabilities such as ACID transactions,** SQL-like queries, indexes, and much more.

DataStore

- Use a managed, NoSQL, **schemaless database for storing non-relational data.**
- Cloud Datastore **automatically scales as you need it and supports transactions as well as robust, SQL-like queries-GQL(Graph Query Language)**

- The underlying infrastructure of **DataStore is based on Bigtable , a redundant, distributed, and semistructured data store that organizes data in the form of tables**

MemCache

- AppEngine provides caching services by means of MemCache.
- This is a distributed in-memory cache that is optimized for fast access and provides developers with **a volatile store for the objects that are frequently accessed.**
- **The caching algorithm implemented by MemCache will automatically remove the objects that are rarely accessed.**

MemCache

- The use of MemCache can significantly **reduce the access time to data;**
- Developers can structure their applications so that **each object is first looked up into MemCache and if there is a miss, it will be retrieved from DataStore and put into the cache for future lookups**

Mail and instant messaging

- To facilitate the implementation of such tasks, AppEngine provides developers with the ability to send and receive mails through **Mail**.
- The service **allows sending email on behalf of the application to specific user accounts.**
- It is also possible to include several types of attachments and to target multiple recipients.
- Mail operates asynchronously, and **in case of failed delivery the sending address is notified through an email detailing the error.**

XMPP

- AppEngine provides also another way to communicate with the external world
- **Extensible Messaging and Presence Protocol (XMPP).**
- **XMPP is the open standard for instant messaging and presence information.**
- XMPP is based on XML (Extensible Markup Language).

XMPP

- A set of open technologies for **instant messaging, presence, multi-party chat, voice and video calls**, collaboration, lightweight middleware, content syndication, and **generalized routing of XML data.**

XMPP

- Any chat service that supports XMPP, such as Google Talk, **can send and receive chat messages to and from the Web application,** which is identified by its own address.
- Even though the chat is a communication medium mostly used for human interactions, **XMPP can be conveniently used to connect the Web application with chat bots** or to implement a small administrative console.

Account management

- Web applications often keep various data that customize their interaction with users.
- These data normally go under the user profile and are attached to an account.
- AppEngine simplifies account management by allowing **developers to leverage Google account management by means of Google Accounts.**

Account management

- Using Google Accounts, **Web applications can conveniently store profile settings in the form of key-value pairs, attach them to a given Google account**, and quickly retrieve them once the user authenticates.
- With respect to a custom solution, **the use of Google Accounts requires users to have a Google account, but it does not require any further implementation.**

Account management

- The use of Google Accounts is particularly **advantageous for developing Web applications within a corporate environment using Google Apps.**
- In this case, the applications **can be easily integrated with all the other services (and profile settings) included in Google Apps**

Image manipulation

- Web applications render pages with graphics. Often simple operations, **such as adding watermarks or applying simple filters, are required.**
- AppEngine allows applications to perform **image resizing, rotation, mirroring, and enhancement by means of Image Manipulation, a service that is also used in other Google products.**
- Image Manipulation is mostly designed for lightweight image processing and is optimized for speed

Google App Engine (GAE)

Google App Engine Front Page: http://code.google.com/appengine/

Signing up for an account or use your gmail account name : https://appengine.google.com/

Downloading GAE SDK : http://code.google.com/appengine/downloads.html

Python Getting Started Guide: http://code.google.com/appengine/docs/python/gettingstarted/

Java Getting Started Guide: http://code.google.com/appengine/docs/java/gettingstarted/

Quota page for free service: http://code.google.com/appengine/docs/quotas.html#Resources

Billing page if you go over the quota:
http://code.google.com/appengine/docs/billing.html#Billable_Quota_Unit_Cost

GAE Applications

- Well-known GAE applications include the Google Search Engine, Google Docs, Google Earth, and Gmail.
- These applications can support large numbers of users simultaneously.
- The applications are all run in the Google data centers. Inside each data center, there might be thousands of server nodes to form different clusters. Each cluster can run multipurpose servers.
- GAE also provides Google-specific services, such as the Gmail account service (which is the login service, that is, applications can use the Gmail account directly).

Cron Jobs

- The App Engine Cron Service **allows you to configure scheduled tasks that operate at defined times or regular intervals.** These tasks are commonly known as cron jobs.
- **These cron jobs are automatically triggered by the App Engine Cron Service.**
- For instance, you might use this to send out a report email on a daily basis, to update some cached data every 10 minutes, or to update some summary information once an hour.

Urlfetch Service

- This service allows scripts to access other resources on the web by fetching URLs.
- A script can use the UrlFetch service to issue HTTP and HTTPS requests and receive responses.
- The UrlFetch service uses Google's network infrastructure for efficiency and scaling purposes.

Class Name	Brief description
HTTPResponse	This class allows users to access specific information on HTTP responses.
UrlFetchApp	Fetch resources and communicate with other hosts over the Internet.